When Your CNN Is a Coin Flip: What a Georgia Tech Practicum Taught Me About Bitcoin Signal Engineering

The seductive hypothesis, the humbling null result, and the simple signal hiding in plain sight

---

I spent the first half of my Georgia Tech OMSA Practicum convinced that convolutional neural networks could see patterns in Bitcoin candlestick charts that humans couldn't. I built a pipeline that converted OHLCV price data into Gramian Angular Field images, trained a CNN to classify directional moves, and fed those predictions into a tournament-style accumulation strategy.

The CNN achieved a 41.43% Rolling Window percentile on the sats-per-dollar metric.

The neutral baseline — a model that literally predicts 0.5 every single day, the equivalent of flipping a coin — scored 41.94%.

My sophisticated deep learning pipeline performed *worse than doing nothing*.

This is the story of what happened next.

The Setup: Why Images?

The Practicum is a capstone project in Georgia Tech's Online Master of Science in Analytics program. My team partnered with the Trilemma Foundation to build a Bitcoin accumulation signal — not a trading strategy in the traditional sense, but a system that decides *how aggressively to buy* on any given day. The evaluation metric is sats-per-dollar: how many satoshis your strategy accumulates per dollar spent, benchmarked against a universe of alternative weight paths.

The original thesis was compelling on paper. Industry reports had shown high F1 scores on candlestick pattern classification tasks. If a CNN could reliably identify bullish vs. bearish formations, we could translate those predictions into allocation weights. Buy more on bullish days, less on bearish ones. Simple.

We built a rigorous pipeline: Gramian Angular Field encoding of OHLCV data into 2D images, a CNN classifier trained on 2014–2015 data, tested out-of-sample on 2016–2025. Deterministic seeds. No lookahead bias. Clean train/test separation.

And it didn't work.

The Autopsy: Why the CNN Failed

The temptation after a null result is to blame the model and try a bigger one. We resisted that. Instead, we ran ablations.

Ablation #1 — Allocator Sensitivity: We tested whether the problem was in how we converted CNN predictions into weights. It wasn't. The allocator behaved correctly with synthetic signals. The problem was upstream.

Ablation #2 — Signal Quality: We examined the CNN's actual predictions. They showed almost no discriminative power. The model was producing probabilities clustered tightly around 0.5 — it had learned to hedge, not to predict.

The root causes became clear:

1. **Time horizon mismatch**: Daily candlestick patterns encode intraday structure. The accumulation tournament evaluates over months and years. Pattern recognition trading edge at the relevant horizon.
2. **Regime shift**: Training on 2014–2015 (pre-institutional Bitcoin) and testing on 2016–2025 (a fundamentally different market) meant the model was learning patterns from a world that no longer existed.
3. **Spatial vs. temporal confusion**: GAF images encode temporal correlations as spatial patterns, but the CNN treats them as generic image features. The conversion loses the sequential structure that matters most.

We presented this as a "coin flip" result at midterm. The graders gave us 88.75/100 — a respectable score, partly because we'd been honest about failure and rigorous in our diagnosis. But I wanted better.

The Pivot: What Was Already There

Here's what I didn't expect: the signal I needed was already sitting in the feature engineering code, waiting to be used differently.

Instead of trying to *classify* market direction, I shifted to *measuring* it. The approach:

1. Fit a rolling OLS regression to log-price over a 60-day window
2. Extract the slope coefficient as a trend indicator
3. Normalize it with a 252-day rolling z-score (regime-relative scaling)
4. Clip at ± 2 to prevent outlier distortion
5. Convert to allocation probability: `prob_up = 0.5 - 0.15 × tanh(z_score)`

That's the entire signal. One line of math after the preprocessing.

The key insight is in step 3. The z-score normalization divides the raw trend by its own recent volatility. In a calm market, a small trend gets amplified. In a volatile market, even a large trend gets dampened. The signal automatically adapts to the current regime without any explicit regime detection.

I call this **regime-relative dampening**. It's not contrarian in the classical sense — the signal actually shows positive correlation with forward returns (r +0.14 at 30 days). It's closer to momentum, but with a built-in volume knob that turns down during chaos and turns up during calm.

The Experiment Discipline

After the CNN failure, I committed to a strict decision framework for every subsequent experiment:

> **Adopt only if**: RW percentile improves AND either (a) win rate does not degrade materially OR (b) mean sats-per-dollar advantage improves.

This sounds obvious. It isn't. When you've been staring at a failing system for weeks, the temptation to adopt any improvement — even a noisy one — is enormous.

Here's the experiment ledger:

Experiment	RW Impact	Decision	Reason
OLS z-score (±2 clip)	+3.01 pp	**ADOPT**	Clear improvement, regime-adaptive
Turnover governor	+0.00 pp	**ADOPT**	~12% turnover reduction, operational stability
Vol-amplitude scaling	+0.14 pp	**REJECT**	Win rate degraded, SPD advantage shrank
Weekly granularity	-0.90 pp	**REJECT**	Worse than daily; gain was from z-window, not resampling
Fee sensitivity	N/A	**DOCUMENT**	Proved percentile-invariant (proportional fees cancel in percentile ratios)

The vol-amplitude experiment was the most instructive rejection. It improved RW by 0.14 percentage points — barely above noise — while degrading win rate by 1.59% and shrinking the absolute sats-per-dollar advantage. The mechanism was clear: the OLS z-score already normalizes by rolling standard deviation.

Adding a second layer of volatility adjustment was redundant. The implicit adaptation was doing the work; the explicit layer just added noise.

The weekly granularity test was the most *surprising* rejection. I'd estimated it could add 10–20 pp based on the logic that weekly data would filter daily noise and align better with Bitcoin's multi-month cycles. The direct weekly equivalent (12/26/52-week windows) actually underperformed by 0.90 pp. The slight improvement from one configuration (12/26/26-week) came from shortening the z-score window, not from weekly resampling itself. The daily signal was already near-optimal.

The Performance Ladder

Configuration	RW Percentile	Win Rate
Neutral (constant prob=0.5)	41.94%	70.42%
OLS raw (no normalization)	43.25%	91.48%
OLS z-score winsorized	**44.95%**	**66.94%**

A +3.01 percentage point improvement over doing nothing. Not a moonshot. Not a 10x return. A modest, defensible edge built on the simplest possible signal architecture.

The Statistical Backbone

For the final report, I added formal statistical tests to validate the regime hypothesis:

- **Ljung-Box Q-statistic** ($Q = 847.3$, $p < 0.001$): Squared returns show significant autocorrelation — volatility clusters, confirming that regime structure exists in the data.
- **Mann-Whitney U test** ($U = 2{,}341{,}887$, $p < 0.001$): Return distributions in bull and bear regimes are statistically different — the regimes aren't just labels, they represent genuinely distinct market behavior.
- **Bootstrap 95% confidence intervals**: Bull regime averages +0.23%/day, bear regime averages -0.09%/day, with non-overlapping confidence intervals.

These tests don't prove the signal works in the future. They prove that the mechanism the signal exploits — volatility-regime structure in Bitcoin returns — is real and statistically significant in the historical data.

What I Actually Learned

1. Pattern recognition trading edge. A CNN can classify candlestick patterns with high accuracy and still be useless for accumulation. The gap

between "correctly identified a hammer pattern" and "this information improves dollar-cost averaging" is enormous.

2. Null results are results. The CNN failure wasn't a waste — it was the most important finding of the project. It eliminated an entire class of approaches and forced a pivot that produced something better.

3. Simplicity isn't a compromise. The final signal is one OLS regression, one z-score, one tanh transform. It outperforms a deep learning pipeline by 3.5 percentage points. The simplicity *is* the feature — fewer parameters means less to overfit.

4. Experiment discipline prevents self-deception. Without the "adopt only if" decision rule, I would have kept vol-amplitude scaling because +0.14 pp *feels* like progress. The rule forced me to look at the full picture.

5. The z-score is doing more than you think. Dividing by rolling standard deviation doesn't just normalize — it implicitly creates regime awareness. In calm markets, signals speak louder. In volatile markets, signals are muted. This is exactly the behavior you want for a long-horizon accumulation strategy.

The Honest Conclusion

I didn't build a system that crushes buy-and-hold. I built a system that modestly improves dollar-cost averaging through regime-aware signal dampening, validated it with formal statistical tests, and documented every failed experiment along the way.

If I've learned one thing from this project, it's that the most important skill in quantitative finance isn't building complex models. It's knowing when to stop building and start measuring — and being honest about what the measurements say.

Explore the Full Analysis: - **GitHub Repository**: bitcoin-analytics-capstone-template — complete codebase, data pipeline, and reproducibility package (30-second smoke test: `make all`) - **Video Walkthrough**: Bitcoin — Win by Failing First — 7-minute explainer for non-technical audiences

---

Bob Katz is a graduate student in Georgia Tech's Online Master of Science in Analytics program and the founder of FACTS Consulting. This article describes work completed for the OMSA Practicum capstone in partnership with the Trilemma Foundation. The code, data, and full experimental log are available in the project repository.